

Using Docker and Docker-Compose

1. Overview

In this lab, you are going to learn several tools that you will be using in subsequent labs and the practical quiz. The lab tasks include:

- a) Using docker and docker compose. Compose is a tool for defining and running multi-container Docker applications¹. It use a YAML configuration file to define your application's services.
- b) Set up Nginx web server
- c) MySQL DB,
- d) a local Git server,
- e) Selenium Server for testing

2. Outcomes

Upon completion of this session, you and your team should be able to

- Understand the basic structure of the Docker compose configuration file
- Define a multi-container docker application with Nginx, MySQL, Git server and Selenium server

¹ <https://docs.docker.com/compose/>

3. Lab

1. The lab requires docker engine and docker compose installed on your laptop. If you are using Docker Desktop, docker compose is already installed with Docker Desktop.
2. The lab is divided into the following sections:
 - i. Understanding docker compose and its compose file specifications
 - ii. Nginx setup with docker compose
 - iii. Mysql setup with docker compose
 - iv. Git server setup with docker compose
 - v. Selenium server setup with docker compose

4. Installation of Docker Desktop

You can use Docker Desktop on your laptop for your individual development work.

Windows 11

1. Docker desktop would require WSL 2 (Windows Subsystem for Linux). Install WSL 2 using: `wsl --install`
1. Next download Docker Desktop (.exe or Windows installer) from <https://www.docker.com/products/docker-desktop/> and install the file.

Mac OS

1. Download Docker Desktop (.dmg or Mac OS installer) from <https://www.docker.com/products/docker-desktop/> and install the file.

Linux

1. For your team VM, you will be using Linux. And you should install Docker Engine on the VM instead. Please follow the installation instructions at: <https://docs.docker.com/engine/install/ubuntu/>
1. The main steps in include:
 - i. Installing Dependencies such as ca-certificates, curl etc
 - ii. Adding Docker Repository
 - iii. Installing the Docker Engine packages
 - iv. Enabling the Docker system service
 - v. Verifying the installation

5. Compose File

1. Docker compose is relatively easy to use. You define a compose yaml file and run:
 - i. `docker-compose up` (windows)
 - ii. or, `sudo docker-compose up` (in mac/linux)
 - iii. To stop container, do `ctrl-c`
 - `docker-compose stop`
 - iv. To stop and remove containers,
 - `docker-compose down`

2. The important part of learning to use compose is to understand the fundamentals of

docker and the compose file specifications. The Compose file is a YAML file defining services, networks, and volumes for a Docker application. The latest and recommended version of the Compose file format is defined by the Compose Specification². Microservices is a fast changing technology and you should visit the footnote to learn about the up-to-date specification.

3. The compose file is named `compose.yml` and includes the following main elements:

- i. Version top-level element. Deprecated.
- ii. Name top-level element. Use `compose.yaml` (preferred) or `compose.yml` in working directory. Older `docker-compose.yaml` and `docker-compose.yml` also supported.
- iii. Services top-level element. Computing components of the application. This is the core of the compose file and contains many other configurable items.
- iv. Network configuration. Services communicate through network.
- v. Volume configuration. Data is persisted to volumes.
- vi. Configs items. Grants access to configs `o/usr/bin` on a per-service basis.
- vii. Secret items. Sensitive data e.g. passwords.

6. Key Networking Concepts in Docker Compose

1. Default Network:

- i. By default, Docker Compose creates a network named `<project_name>_default` for services in a Compose file. All services in the Compose file are connected to this network unless explicitly configured otherwise.

² <https://docs.docker.com/compose/compose-file/>
ICT2216/ICT2516C Secure Software Development

2. Custom Networks:

- i. You can define custom networks to better control service communication.

Networks can be of different types:

- **bridge** (default for standalone containers)
- **host** (removes network isolation; uses host networking)
- **none** (disables networking)

3. Aliases:

- i. You can assign a custom hostname alias to a service within a specific network for easier communication.

A sample docker network configuration using docker-compose is shown below:

```
services:
  app:
    image: myapp:latest
    networks:
      - my_network
    ports:
      - "8080:80"
networks:
  my_network:
    driver: bridge
```

7. Nginx

Step 1: Create the following compose.yaml file, in your lab / project folder

```
services:
  nginxwebsvr:
    image: nginx:alpine
    container_name: nginxwebsvr
    ports:
      - "80:80"
```

Step 2: In command prompt run:

```
docker-compose up
```

Step 3: access <http://localhost/>

```
docker ps -all
```

8. MySQL

Step 1: Create mysql data directory in project folder

```
mkdir ./mysql_data
```

Step 2: Update the compose.yaml file

```
services:
  nginxwebsvr:
    image: nginx:alpine
    container_name: nginxwebsvr
    ports:
      - "80:80"
  mysqldb:
    image: mysql:8.0
    restart: always
    volumes:
      - ./mysql_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: pass
      MYSQL_DATABASE: testdb
      MYSQL_USER: user
      MYSQL_PASSWORD: pass
```

Step 3: In command prompt run:

```
docker-compose up
```

Step 4: Check mysql:

```
docker ps -all
docker exec -it <mysqldb-cntr-id> /bin/sh
cd /usr/bin; ./mysql -u root -p
```

9. Git Server

To run a git based SCM tool using docker, you can use popular tools such as GitLab or Gitea. Alternatively, you can install git on a light-weight linux image. The git server can be setup to have no authentication (NOT recommended), ssh access using keys or https access using cert.

For this lab purpose, we will set up a git-server without any authentication. Additional resources to configure ssh access using keys is provided next.

9.1. Git Server without any Authentication

Step 1: Create your repo directory (on the host)

```
mkdir ./repos/
```

Step 2: Create Dockerfile, named gitserver.Dockerfile

```
FROM node:alpine
```

```
RUN apk add --no-cache tini git \  
    && yarn global add git-http-server \  
    && adduser -D -g git git
```

```
USER git  
WORKDIR /home/git
```

```
RUN git init --bare repository.git
```

```
ENTRYPOINT ["tini", "--", "git-http-server", "-p", "3000", "/home/git"]
```


Step 2: Add the following service to your compose.yaml file

```
services:
  git-server:
    build:
      dockerfile: gitserver.Dockerfile
      context: .
    restart: always
    ports:
      - "3000:3000"
    volumes:
      - ./repos:/var/www/git
```

Step 3: docker-compose build

Step 4: docker-compose up -d

Step 4: Testing:

```
git clone http://localhost:3000/repository.git
```

9.2. Git Server with SSH Access (Optional)

Step 1: Generate ssh key under `./keys/`

Step 2: Update your dockerfile to include:

- i. openssh installation
- ii. set container work directory to `/git-server/`
- iii. add volume, `./keys` to `/git-svr/keys/`
- iv. configure git user ssh. This include adding the authorized key file, setting the right permission for ssh keys and turning off password ssh access

Step 3: It can be tedious to repeat the above during container restarts. So you could use scripts to automate them. There are opensource, public projects³ ⁴ that you can reference on this.

³ <https://github.com/6arms1leg/git-ssh-docker>

⁴ <https://github.com/jkarlosb/git-server-docker/blob/master/Dockerfile>

10. Selenium Server

Selenium is a useful tool for automating web browser interactions and is widely used for automated web app testing. The Selenium Server acts as a central hub that allows multiple Selenium WebDriver sessions to run on different browsers and machines. It is also used for remote execution and parallel testing.

Step 1: The docker-compose configuration is placed in the compose-selenium.yaml file

Step 2: Start the docker-compose:

```
sudo docker-compose -f compose-selenium.yaml up
```

Step 3: Check the server status:

```
curl http://localhost:4444/wd/hub/status
```

You should see the appropriate selenium status output.

END OF DOCUMENT